

Android XML 原生AlertDialog详解（基础+自定义）

在 Android 开发中，AlertDialog（警告对话框）是用于与用户进行短期交互的核心组件，常用于显示提示信息、确认操作、收集简单输入等场景。它具有模态特性，弹出后会阻断当前界面的交互，直到用户做出响应（如点击按钮）。本文从「原生基础用法」到「深度自定义布局与样式」全面拆解，搭配可直接运行的代码示例，帮助开发者快速掌握AlertDialog的使用与定制技巧。

一、核心概念：AlertDialog 是什么？

AlertDialog 继承自 Dialog，是 Android 系统提供的标准化对话框，核心特性如下：

- **模态交互**：弹出后阻断当前界面操作，必须用户响应才能关闭，确保重要信息被关注
- **标准化结构**：原生支持「标题区+内容区+按钮区」的经典结构，按钮区最多支持3个按钮（Positive / Neutral / Negative）
- **样式自适应**：默认跟随系统主题样式，可通过主题属性快速调整，也能自定义布局实现个性化效果
- **功能丰富**：支持显示文本、列表、单选列表、多选列表，还可嵌入自定义视图（如输入框、进度条）

💡 注意：AlertDialog 不能直接通过 new 关键字创建实例，需通过 Builder 构建者模式创建，这是系统推荐的解耦方式，便于灵活配置对话框属性。

二、基础用法：原生 AlertDialog（4类核心场景）

原生 AlertDialog 的使用核心是「AlertDialog.Builder 配置 + 方法调用」，以下按开发中最常用的场景分类讲解，所有示例均基于 Kotlin 编写，Java 开发者可参考语法转换。

场景 1：基础确认对话框（文本内容+按钮）

最常用场景，用于提示信息并获取用户确认（如「是否删除该文件？」），包含标题、内容和两个操作按钮。

步骤 1：通过 Builder 配置对话框

Code block

```
1 import android.os.Bundle
```

```

2  import android.view.View
3  import android.widget.Button
4  import android.widget.Toast
5  import androidx.appcompat.app.AlertDialog
6  import androidx.appcompat.app.AppCompatActivity
7
8  class MainActivity : AppCompatActivity() {
9      override fun onCreate(savedInstanceState: Bundle?) {
10         super.onCreate(savedInstanceState)
11         setContentView(R.layout.activity_main)
12
13         // 绑定触发按钮（布局中需添加一个Button，id为btn_show_basic）
14         val btnShowBasic = findViewById<Button>(R.id.btn_show_basic)
15         btnShowBasic.setOnClickListener {
16             showBasicConfirmDialog()
17         }
18     }
19
20     // 显示基础确认对话框
21     private fun showBasicConfirmDialog() {
22         // 1. 创建Builder实例，关联上下文
23         val builder = AlertDialog.Builder(this)
24         // 2. 配置对话框属性
25         builder.apply {
26             setTitle("删除确认") // 设置标题
27             setMessage("确定要删除该文件吗？删除后无法恢复。") // 设置内容文本
28             setIcon(R.drawable.ic_warning) // 设置图标（可选，如警告图标）
29
30             // 3. 设置确认按钮（PositiveButton：通常表示"确定"，右对齐）
31             setPositiveButton("确定") { dialog, which ->
32                 // 点击确认后的逻辑（如执行删除操作）
33                 Toast.makeText(this@MainActivity, "文件已删除",
34                     Toast.LENGTH_SHORT).show()
35                 dialog.dismiss() // 关闭对话框（按钮点击后默认会关闭，可省略）
36             }
37
38             // 4. 设置取消按钮（NegativeButton：通常表示"取消"，左对齐）
39             setNegativeButton("取消") { dialog, which ->
40                 // 点击取消后的逻辑（如取消操作）
41                 Toast.makeText(this@MainActivity, "已取消删除",
42                     Toast.LENGTH_SHORT).show()
43                 dialog.dismiss()
44             }
45
46             // 5. 设置中立按钮（NeutralButton：可选，通常表示"稍后"，居中对齐）
47             setNeutralButton("稍后提醒") { dialog, which ->

```

```

46         Toast.makeText(this@MainActivity, "已设置稍后提醒",
Toast.LENGTH_SHORT).show()
47         dialog.dismiss()
48     }
49
50     // 6. 设置对话框是否可通过点击外部关闭 (默认true, false表示必须点击按钮关
闭)
51     setCancelable(false)
52 }
53
54 // 7. 构建并显示对话框
55 val dialog = builder.create()
56 dialog.show()
57 }
58 }

```

步骤 2: 布局文件 (activity_main.xml)

Code block

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent"
6      android:orientation="vertical"
7      android:padding="20dp"
8      android:gravity="center">
9
10     <Button
11         android:id="@+id/btn_show_basic"
12         android:layout_width="wrap_content"
13         android:layout_height="wrap_content"
14         android:text="显示基础确认对话框"
15         android:textSize="16sp"/>
16
17 </LinearLayout>

```

场景 2: 列表对话框 (单选/多选)

用于让用户从预设列表中选择选项，分为「单选列表」和「多选列表」两种，适用于如「选择支付方式」「批量删除文件」等场景。

1. 单选列表对话框

```

1 // 显示单选列表对话框
2 private fun showSingleChoiceDialog() {
3     val paymentMethods = arrayOf("支付宝", "微信支付", "银行卡支付", "现金支付")
4     // 列表数据
5     var selectedIndex = 0 // 默认选中索引 (第1项)
6     AlertDialog.Builder(this)
7         .setTitle("选择支付方式")
8         .setIcon(R.drawable.ic_payment)
9         // 设置单选列表: 参数为 (列表数据, 默认选中索引, 选择监听)
10        .setSingleChoiceItems(paymentMethods, selectedIndex) { dialog, which ->
11            selectedIndex = which // 更新选中索引
12            // 可选: 点击选项后立即关闭对话框 (如不需要实时反馈可省略)
13            // dialog.dismiss()
14        }
15        .setPositiveButton("确认") { dialog, which ->
16            // 点击确认后获取选中结果
17            val selectedMethod = paymentMethods[selectedIndex]
18            Toast.makeText(this, "已选择: $selectedMethod",
19                Toast.LENGTH_SHORT).show()
20        }
21        .setNegativeButton("取消", null) // 取消按钮, 点击后默认关闭对话框
22        .show()
23    }
24 }

```

2. 多选列表对话框

```

Code block
1 // 显示多选列表对话框
2 private fun showMultiChoiceDialog() {
3     val fileNames = arrayOf("文件1.txt", "文件2.png", "文件3.docx", "文件4.pdf")
4     // 列表数据
5     val isSelected = booleanArrayOf(true, false, true, false) // 默认选中状态 (第
6     1、3项选中)
7     AlertDialog.Builder(this)
8         .setTitle("选择要删除的文件")
9         .setIcon(R.drawable.ic_file)
10        // 设置多选列表: 参数为 (列表数据, 默认选中状态, 选择监听)
11        .setMultiChoiceItems(fileNames, isSelected) { dialog, which, isChecked
12            ->
13                isSelected[which] = isChecked // 更新选中状态
14        }
15        .setPositiveButton("删除所选") { dialog, which ->
16            // 遍历选中状态数组, 获取选中的选项
17        }
18    }
19 }

```

```

15         val selectedFiles = mutableListOf<String>()
16         for (i in isSelected.indices) {
17             if (isSelected[i]) {
18                 selectedFiles.add(fileNames[i])
19             }
20         }
21         Toast.makeText(this, "已删除: ${selectedFiles.joinToString(", ")}",
Toast.LENGTH_SHORT).show()
22     }
23     .setNegativeButton("取消", null)
24     .show()
25 }

```

场景 3：带输入框的对话框

用于收集用户简单输入（如「输入用户名」「设置密码」），原生支持嵌入 EditText，适用于轻量级输入场景。

Code block

```

1  // 显示带输入框的对话框
2  private fun showInputDialog() {
3      // 1. 创建输入框并配置属性
4      val editText = EditText(this).apply {
5          hint = "请输入用户名"
6          textSize = 16f
7          padding = resources.getDimensionPixelSize(R.dimen.dp_15) // 配置内边距
8          layoutParams = ViewGroup.LayoutParams(
9              ViewGroup.LayoutParams.MATCH_PARENT,
10             ViewGroup.LayoutParams.WRAP_CONTENT
11         )
12     }
13
14     // 2. 构建对话框并嵌入输入框
15     AlertDialog.Builder(this)
16         .setTitle("设置用户名")
17         .setIcon(R.drawable.ic_edit)
18         .setView(editText) // 将输入框设置为对话框内容
19         .setPositiveButton("确认") { dialog, which ->
20             val username = editText.text.toString().trim()
21             if (username.isEmpty()) {
22                 Toast.makeText(this, "用户名不能为空", Toast.LENGTH_SHORT).show()
23                 return@setPositiveButton
24             }
25             Toast.makeText(this, "用户名已设置为: $username",
Toast.LENGTH_SHORT).show()

```

```

26         }
27         .setNegativeButton("取消", null)
28         .show()
29     }

```

注：需在 res/values/dimens.xml 中添加内边距配置： `<dimen name="dp_15">15dp</dimen>`

场景 4：进度对话框（加载中提示）

用于显示耗时操作的进度（如文件下载、数据加载），分为「不确定进度」和「确定进度」两种，原生通过 ProgressDialog 实现（AlertDialog 可间接集成）。

Code block

```

1  // 显示确定进度的对话框（如文件下载）
2  private fun showProgressDialog() {
3      val progressDialog = ProgressDialog(this).apply {
4          setTitle("文件下载中")
5          setMessage("请稍候...")
6          setIcon(R.drawable.ic_download)
7          setProgressStyle(ProgressDialog.STYLE_HORIZONTAL) // 水平进度条
8          max = 100 // 进度最大值
9          setCancelable(false) // 加载中不允许取消
10     }
11
12     progressDialog.show()
13
14     // 模拟进度更新（实际开发中需在子线程中执行）
15     Thread {
16         for (progress in 0..100 step 10) {
17             Thread.sleep(500) // 模拟耗时操作
18             // 更新进度需在主线程
19             runOnUiThread {
20                 progressDialog.progress = progress
21                 if (progress == 100) {
22                     progressDialog.dismiss()
23                     Toast.makeText(this@MainActivity, "下载完成",
24                         Toast.LENGTH_SHORT).show()
25                 }
26             }
27         }.start()
28     }

```

三、核心 API 与属性详解

AlertDialog 的功能通过 Builder 类的方法配置，以下是开发中高频使用的 API 分类说明：

1. 基础配置方法

方法名	作用	参数说明
setTitle(CharSequence)	设置对话框标题	标题文本（支持 Spanned 富文本）
setMessage(CharSequence)	设置对话框内容文本	内容文本（支持换行符 \n）
setIcon(Int)	设置对话框图标	图标资源 id（如 R.drawable.ic_warning）
setCancelable(Boolean)	设置是否可通过外部点击关闭	true（默认） / false（强制按钮关闭）
setOnCancelListener(DialogInterface.OnCancelListener)	监听对话框取消事件	取消时的回调（如点击外部或返回键）

2. 按钮配置方法

方法名	作用	特点
setPositiveButton(CharSequence, DialogInterface.OnClickListener)	设置确认按钮	右对齐，通常表示「确定」「完成」
setNegativeButton(CharSequence, DialogInterface.OnClickListener)	设置取消按钮	左对齐，通常表示「取消」「返回」
setNeutralButton(CharSequence, DialogInterface.OnClickListener)	设置中立按钮	居中对齐，通常表示「稍后」「帮助」

3. 内容区配置方法

方法名	作用	适用场景
setMessage(CharSequence)	设置纯文本内容	简单提示信息
setSingleChoiceItems(Array , Int, DialogInterface.OnClickListener)	设置单选列表	单一选项选择（如支付方式）
setMultiChoiceItems(Array, BooleanArray, DialogInterface.OnMultiChoiceClickListener)	设置多选列表	批量选项选择（如批量删除）
setView(View)	设置自定义视图	复杂内容（如输入框、表单）

四、进阶：自定义 AlertDialog（布局与样式）

原生 AlertDialog 的样式和结构较为固定，实际开发中常需根据 UI 设计定制对话框（如圆角、渐变背景、复杂表单），核心方案是「自定义布局文件 + 关联到 AlertDialog」。

场景 1：自定义布局对话框（登录表单示例）

需求：实现一个带用户名、密码输入框的登录对话框，包含自定义标题、按钮样式和圆角背景。

步骤 1：创建自定义布局文件（dialog_login.xml）

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="wrap_content"
6      android:orientation="vertical"
7      android:padding="20dp"
8      android:background="@drawable/dialog_bg">
9      <TextView
10         android:id="@+id/tv_login_title"
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:text="用户登录"
```

```
14         android:textSize="20sp"
15         android:textColor="@color/black"
16         android:textStyle="bold"/>
17
18
19     <EditText
20         android:id="@+id/et_username"
21         android:layout_width="match_parent"
22         android:layout_height="50dp"
23         android:layout_marginTop="15dp"
24         android:hint="请输入用户名"
25         android:inputType="text"
26         android:paddingHorizontal="15dp"
27         android:background="@drawable/et_bg_normal"/>
28
29
30     <EditText
31         android:id="@+id/et_password"
32         android:layout_width="match_parent"
33         android:layout_height="50dp"
34         android:layout_marginTop="10dp"
35         android:hint="请输入密码"
36         android:inputType="textPassword"
37         android:paddingHorizontal="15dp"
38         android:background="@drawable/et_bg_normal"/>
39
40
41     <LinearLayout
42         android:layout_width="match_parent"
43         android:layout_height="wrap_content"
44         android:layout_marginTop="20dp"
45         android:orientation="horizontal"
46         android:gravity="end">
47
48         <Button
49             android:id="@+id/btn_cancel"
50             android:layout_width="wrap_content"
51             android:layout_height="40dp"
52             android:text="取消"
53             android:textSize="16sp"
54             android:layout_marginRight="10dp"
55             android:backgroundTint="@color/gray_e0"/>
56
57         <Button
58             android:id="@+id/btn_login"
59             android:layout_width="wrap_content"
60             android:layout_height="40dp"
```

```
61         android:text="登录"
62         android:textSize="16sp"
63         android:backgroundTint="@color/blue"/>
64
65     </LinearLayout>
66
67 </LinearLayout>
```

步骤 2：创建自定义背景资源

1. 对话框圆角背景（res/drawable/dialog_bg.xml）：

Code block

```
1 <shape xmlns:android="http://schemas.android.com/apk/res/android"
2     android:shape="rectangle">
3     <solid android:color="@color/white"/>
4     <corners android:radius="12dp"/>
5     <stroke android:width="1dp" android:color="@color/gray_e0"/>
6 </shape>
```

2. 输入框背景（res/drawable/et_bg_normal.xml，同前文EditText示例）

步骤 3：在代码中加载自定义布局并绑定事件

Code block

```
1 // 显示自定义登录对话框
2 private fun showCustomLoginDialog() {
3     // 1. 加载自定义布局
4     val dialogView = LayoutInflater.from(this).inflate(R.layout.dialog_login,
5         null)
6     // 2. 绑定布局中的控件
7     val etUsername = dialogView.findViewById<EditText>(R.id.et_username)
8     val etPassword = dialogView.findViewById<EditText>(R.id.et_password)
9     val btnCancel = dialogView.findViewById<Button>(R.id.btn_cancel)
10    val btnLogin = dialogView.findViewById<Button>(R.id.btn_login)
11
12    // 3. 构建对话框
13    val dialog = AlertDialog.Builder(this)
14        .setView(dialogView) // 关联自定义布局
15        .setCancelable(false) // 不允许外部关闭
16        .create()
17
18    // 4. 去除对话框默认标题栏（避免与自定义标题冲突）
19    dialog.requestWindowFeature(Window.FEATURE_NO_TITLE)
```

```

19
20 // 5. 绑定按钮事件
21 btnCancel.setOnClickListener {
22     dialog.dismiss()
23 }
24 btnLogin.setOnClickListener {
25     val username = etUsername.text.toString().trim()
26     val password = etPassword.text.toString().trim()
27     when {
28         username.isEmpty() -> Toast.makeText(this, "请输入用户名",
29 Toast.LENGTH_SHORT).show()
30         password.isEmpty() -> Toast.makeText(this, "请输入密码",
31 Toast.LENGTH_SHORT).show()
32         else -> {
33             Toast.makeText(this, "登录成功: $username",
34 Toast.LENGTH_SHORT).show()
35             dialog.dismiss()
36         }
37     }
38 }
39 }
40
41 // 6. 显示对话框并调整宽度（可选，让对话框宽度适配屏幕）
42 dialog.show()
43 dialog.window?.apply {
44     setLayout(
45         resources.getDimensionPixelSize(R.dimen.dp_300), // 宽度
46         ViewGroup.LayoutParams.WRAP_CONTENT // 高度自适应
47     )
48     setBackgroundDrawableResource(android.R.color.transparent) // 去除默认背
49     景
50 }
51 }
52 }

```

注：需在 res/values/dimens.xml 中添加宽度配置： `<dimen name="dp_300">300dp</dimen>`

场景 2：自定义对话框样式（全局主题适配）

若需让所有 AlertDialog 统一样式（如圆角、标题颜色），可通过自定义主题实现，无需逐个配置。

步骤 1：在 styles.xml 中定义自定义主题

Code block

```

1 <resources>
2

```

```
3     <style name="AppTheme" parent="Theme.AppCompat.Light.DarkActionBar">
4         <item name="colorPrimary">@color/blue</item>
5         <item name="colorPrimaryDark">@color/blue_dark</item>
6         <item name="colorAccent">@color/blue</item>
7
8         <item name="alertDialogTheme">@style/CustomAlertDialogTheme</item>
9     </style>
10
11
12     <style name="CustomAlertDialogTheme"
13 parent="Theme.AppCompat.Light.Dialog.Alert">
14
15         <item name="android:windowBackground">@drawable/dialog_bg</item>
16
17         <item name="android:windowTitleStyle">@style/DialogTitleStyle</item>
18
19         <item
20 name="android:textAppearanceMedium">@style/DialogContentStyle</item>
21
22         <item name="buttonBarButtonStyle">@style/DialogButtonStyle</item>
23
24         <item name="android:windowFrame">@null</item>
25
26         <item name="android:windowCornerRadius">12dp</item>
27     </style>
28
29     <style name="DialogTitleStyle"
30 parent="Base.TextAppearance.AppCompat.Title">
31         <item name="android:textSize">20sp</item>
32         <item name="android:textColor">@color/black</item>
33         <item name="android:textStyle">bold</item>
34     </style>
35
36     <style name="DialogContentStyle"
37 parent="Base.TextAppearance.AppCompat.Medium">
38         <item name="android:textSize">16sp</item>
39         <item name="android:textColor">@color/gray_99</item>
40     </style>
41
42     <style name="DialogButtonStyle" parent="Widget.AppCompat.Button">
43         <item name="android:textSize">16sp</item>
44         <item name="android:textColor">@color/blue</item>
45     </style>
46 </resources>
```

步骤 2: 在 AndroidManifest.xml 中应用主题

Code block

```
1  <application
2      android:name=".MyApplication"
3      android:allowBackup="true"
4      android:icon="@mipmap/ic_launcher"
5      android:label="@string/app_name"
6      android:roundIcon="@mipmap/ic_launcher_round"
7      android:supportsRtl="true"
8      android:theme="@style/AppTheme">
9      <activity android:name=".MainActivity">
10         <intent-filter>
11             <action android:name="android.intent.action.MAIN" />
12             <category android:name="android.intent.category.LAUNCHER" />
13         </intent-filter>
14     </activity>
15 </application>
```

此后通过 `AlertDialog.Builder(this)` 创建的对话框，都会自动应用自定义主题样式。

五、高级：封装通用自定义 Dialog 工具类

实际开发中，对话框会在多个页面重复使用，将自定义 Dialog 封装为工具类，可减少代码冗余，提升开发效率。

步骤 1: 创建 Dialog 工具类 (DialogUtils.kt)

Code block

```
1  import android.app.Activity
2  import android.app.AlertDialog
3  import android.view.LayoutInflater
4  import android.view.View
5  import android.view.ViewGroup
6  import android.view.Window
7  import android.widget.Button
8  import android.widget.EditText
9  import android.widget.Toast
10 import com.example.yourpackage.R // 替换为你的包名
11
12 object DialogUtils {
13     // 1. 显示基础确认对话框
```

```

14     fun showConfirmDialog(
15         activity: Activity,
16         title: String,
17         message: String,
18         onConfirm: () -> Unit // 确认回调
19     ) {
20         AlertDialog.Builder(activity)
21             .setTitle(title)
22             .setMessage(message)
23             .setPositiveButton("确认") { dialog, _ ->
24                 onConfirm.invoke()
25                 dialog.dismiss()
26             }
27             .setNegativeButton("取消", null)
28             .show()
29     }
30
31     // 2. 显示自定义登录对话框
32     fun showLoginDialog(
33         activity: Activity,
34         onLoginSuccess: (username: String) -> Unit // 登录成功回调
35     ) {
36         val dialogView =
37             LayoutInflater.from(activity).inflate(R.layout.dialog_login, null)
38         val etUsername = dialogView.findViewById<EditText>(R.id.et_username)
39         val etPassword = dialogView.findViewById<EditText>(R.id.et_password)
40         val btnCancel = dialogView.findViewById<Button>(R.id.btn_cancel)
41         val btnLogin = dialogView.findViewById<Button>(R.id.btn_login)
42
43         val dialog = AlertDialog.Builder(activity)
44             .setView(dialogView)
45             .setCancelable(false)
46             .create()
47
48         dialog.requestWindowFeature(Window.FEATURE_NO_TITLE)
49
50         btnCancel.setOnClickListener { dialog.dismiss() }
51         btnLogin.setOnClickListener {
52             val username = etUsername.text.toString().trim()
53             val password = etPassword.text.toString().trim()
54             when {
55                 username.isEmpty() -> Toast.makeText(activity, "请输入用户名",
56                     Toast.LENGTH_SHORT).show()
57                 password.isEmpty() -> Toast.makeText(activity, "请输入密码",
58                     Toast.LENGTH_SHORT).show()
59                 else -> {
60                     onLoginSuccess.invoke(username)
61                 }
62             }
63         }
64     }

```

```

58         dialog.dismiss()
59     }
60 }
61 }
62
63 dialog.show()
64 dialog.window?.apply {
65     setLayout(
66         activity.resources.getDimensionPixelSize(R.dimen.dp_300),
67         ViewGroup.LayoutParams.WRAP_CONTENT
68     )
69     setBackgroundDrawableResource(android.R.color.transparent)
70 }
71 }
72
73 // 可根据需求扩展其他类型对话框（如进度对话框、输入对话框）
74 }

```

步骤 2：在 Activity 中调用工具类

Code block

```

1  // 调用确认对话框
2  DialogUtils.showConfirmDialog(this, "删除确认", "确定要删除该文件吗? ") {
3      // 确认后的逻辑
4      Toast.makeText(this, "文件已删除", Toast.LENGTH_SHORT).show()
5  }
6
7  // 调用登录对话框
8  DialogUtils.showLoginDialog(this) { username ->
9      // 登录成功后的逻辑
10     Toast.makeText(this, "登录成功: $username", Toast.LENGTH_SHORT).show()
11 }

```

六、常见问题与解决方案

1. 对话框标题与自定义布局冲突

原因：使用 setTitle() 方法后，对话框会显示默认标题栏，与自定义布局中的标题重复。

解决：不调用 setTitle() 方法，直接在自定义布局中添加标题；或调用

```
dialog.requestWindowFeature(Window.FEATURE_NO_TITLE)
```

隐藏默认标题栏。

2. 自定义布局宽度过窄或过宽

原因：对话框宽度默认由内容决定，未主动设置适配屏幕。

解决：对话框 `show()` 后，通过 `window.setLayout()` 手动设置宽度，如

```
setLayout(resources.getDimensionPixelSize(R.dimen.dp_300),  
ViewGroup.LayoutParams.WRAP_CONTENT)。
```

3. 对话框背景有黑边或默认阴影

原因：未去除对话框默认背景，与自定义圆角背景冲突。

解决：通过

```
dialog.window?.setBackgroundDrawableResource(android.R.color.transparent)  
去除默认背景。
```

4. 屏幕旋转后对话框消失

原因：屏幕旋转会导致 Activity 重建，未保存对话框状态。

解决：在 `AndroidManifest.xml` 对应 Activity 中添加属性

```
android:configChanges="orientation|screenSize"，禁止 Activity 重建；或通过  
onSaveInstanceState 保存对话框状态。
```

5. 对话框中的 EditText 无法获取焦点

原因：对话框未正确关联软键盘，或 EditText 未设置可编辑属性。

解决：确保 EditText 的 `android:enabled` 和 `android:focusable` 为 `true`；对话框显示后调用

```
etUsername.requestFocus() 主动获取焦点，并弹出软键盘：
```

Code block

```
1 // 弹出软键盘  
2 val imm = getSystemService(INPUT_METHOD_SERVICE) as InputMethodManager  
3 imm.showSoftInput(etUsername, InputMethodManager.SHOW_IMPLICIT)
```

6. 主题样式不生效

原因：自定义主题未正确继承父主题，或未在 `AndroidManifest.xml` 中应用主题。

解决：确保自定义主题父类为 `Theme.AppCompat.Light.Dialog.Alert` 等系统对话框主题；在 `Application` 或 `Activity` 中应用自定义主题。

七、总结

`AlertDialog` 是 Android 开发中不可或缺的交互组件，其使用需根据场景选择合适的实现方式：简单提示用原生文本对话框，选项选择用列表对话框，复杂表单用自定义布局对话框。通过封装工具类和统

一主题样式，可大幅提升开发效率和界面一致性。

本文覆盖了 AlertDialog 从基础到高级的全场景使用需求，开发者可根据实际业务扩展工具类中的方法。若需要实现更特殊的对话框（如带倒计时的确认框、底部弹出的对话框），可基于本文思路进一步定制。

（注：文档部分内容可能由 AI 生成）